Math 118 Final Exam Knowledge Check List March 10, 2020

Here's what you need to know for the final. In short, you need to know everything up to and including Logistic Regression. There are a few things (for example, Viterbi's algorithm) that we didn't cover very thoroughly in class, so these will not be examined. I will indicate these topics clearly below. I encourage you to use this list as a checklist; for each item listed below ask yourself if you understand what it means, and refer back to the lecture notes as necessary.

1. Matrices, Vectors and Norms

- Vector Norms:
  - Definitions.
  - Computing p-norms (note these are also called ℓ_p norms).
  - Cauchy-Schwarz inequality.
- Matrix norms:
  - Notion of induced matrix norm.
  - Computing $\|A\|_1, \|A\|_2$ and $\|A\|_\infty$ as well as the Frobenius norm; $\|A\|_F$.
- Matrix Multiplication:
  - Inner and outer product interpretation. These were covered in HW3.
  - Rank, and its relation to the outer product interpretation of matrix multiplication.
- Linear Systems:
  - Solving Linear Systems using the LU decomposition.
  - Finding the LU decomposition, using at most one pivot step.
  - Condition number and its affect on the solution to linear systems.
- Least Squares:
  - Motivation. That is, why use least squares instead of solving linear system.
  - Solving least squares using normal equations. Pros and cons of this approach.
  - Solving least squares with QR decomposition. Pros and cons of this approach.
  - Properties of orthogonal matrices. Particularly which norms are unitarily invariant.
  - Perturbation analysis for least squares.
- The Singular Value Decomposition:
  - Definitions and properties of singular values and singular vectors.
  - Relation between SVD and outer product interpretation of matrix multiplication. Best rank r approximation of a matrix.
  - Using SVD for PCA. Interpreting the results.
- Tensors:
  - Definitions (*i.e.* what is a tensor, what is a mode and so on).
  - Some uses of tensors.
  - Unfolding of tensors.
  - Multiplication of tensors.
  - **Note:** I will **not** test you on the HOSVD.
- k-means:
  - When to use k-means (*i.e.* what is a clustering problem).

  - implementing k-means: either pseudocode or running an iteration or two on a toy data set.
  - The Quality score, or function that k-means implicitly seeks to minimize.
  - When not to use k-means (*i.e.* what sort of data sets it is likely to fail on).
- Non-negative matrix factorization:
  - Definition.
  - Setting this up as an optimization problem.
  - Using this to extract relevant topics from text data, segment customers etc.
- Graphs:
  - Basic definitions (*i.e.* vertices/nodes, edges, adjacency matrix etc.
  - Intuitive notion of what a cluster is.
  - Spectral Clustering:
    * Heuristic motivation for spectral clustering in terms of matrix perturbation.
    * Cut, Ratio Cut and its relation to spectral clustering.
    * Implementing spectral clustering: either pseudocode or performing it on a small data set, given the eigenvectors.
  - **Note:** I will **not** be testing you on centrality measures.
- Classifying Handwritten Digits
  - Being able to describe and/or recognize a classification problem.
  - Understanding how images can be converted to matrices.
  - the k-NN classifier:
    * Basic idea and justification.
    * Implementing this algorithm, either pseudocode or a toy example.
    * Pros and cons.
  - The SVD bases classifier:
    * Idea behind this, in particular why we might expect images in the same class to live in a low dimensional space.
    * Implementation: I won't ask you for pseudocode or to do a toy example, but make sure you understand all the steps and are ready to fill in the gaps if I give you partially completed pseudocode.
    * The definition of a projection matrix.
    * Pros and cons.
- Text pre-processing and Text retrieval:
  - I won't test you on tokenizing, stemming, lemmatizing etc. but you should at least have some idea of what sort of preprocessing needs to be done on text documents.
  - tf-idf: Know what it is, why its important and how to use it.
  - Non-negative matrix factorization to extract topics from a corpus of documents.
  - **Note:** I will **not** test you on query matching.
- Linear Programming:
  - Converting a word problem to a Linear Program.
  - Definitions: constraints, feasible region, cost function etc.
  - Solving simple LP's graphically.
  - Converting LP's to standard form using slack variables.
- The Simplex Method.

  - Know, and be able to define, what feasible points and basic feasible solutions are.
  - Taking a step of the simplex method, given a basic feasible solution. This might involve computing the edge vectors $\boldsymbol{\delta}_j$, computing the unit costs along the edges, choosing an edge (and knowing when the algorithm terminates), and finding a new basic feasible solution.
  - Understand why the 2-phase simplex method is necessary.
- Convex Optimization.
  - Definitions of convex sets and convex functions.
  - Alternate definitions of convex functions in terms of first and second derivatives.
  - Basic properties of convex functions (*e.g.* composition of convex and convex + non-decreasing is convex.
  - Further definitions: Lipschitz differentiability and strong convexity.
  - Gradient Descent: definition of the iteration, reasons why it works.
  - Newton's Method: definition of the iteration, derivation via the quadratic approximation.
  - Pros and cons of Gradient Descent and Newton's Method, some ways to fix the cons.
- Constrained Optimization.
  - The idea of a constraint.
  - **You do not need to know any of the theory discussed in Lecture 16**.
  - Projected Gradient Descent (PGD): definition of the iteration, the formula for projection in simple cases covered in class.
  - Convergence rate of PGD.
  - **You do not need to know anything about the Lagrange Method**
  - Penalty Method: definition of penalty function, connection to Lasso and Ridge Regression problems.
- Discrete Optimization: What this means and why it is harder than convex optimization.
- Dynamic Programming.
  - Why dynamic programming can be better than brute force.
  - Components of dynamic programming problems.
  - Traceback, and why it is important.
  - Using dynamic programming for Longest Common Subsequence.
  - Using dynamic programming for knapsack problem.
  - **I will not test you on Hidden Markov Models or Viterbi's algorithm.**
- PageRank
  - Set up and motivation for PageRank. Connection to directed graphs.
  - Reducible, and irreducible matrices, fact that Perron-Frobenius guarantees leading eigenvector is strictly positive.
  - teleportation and the matrix $\tilde{P}$.
  - The power method.
  - Putting all of the above together to compute PageRank scores.
- Logistic Regression.
  - Motivation: the Binary Classification problem.
  - The sigmoid function and the logistic regression model.

- I won't ask you to re-derive the Maximum Likelihood Estimator, but you should understand what this is and be ready to fill in gaps.
- The loss function $L(\boldsymbol{\theta}, b)$. The fact that this is convex and why this is important.
- The connection to gradient descent.
- What the classification/decision boundary is, and why it is important.
- Pros and cons of Logistic Regression.